

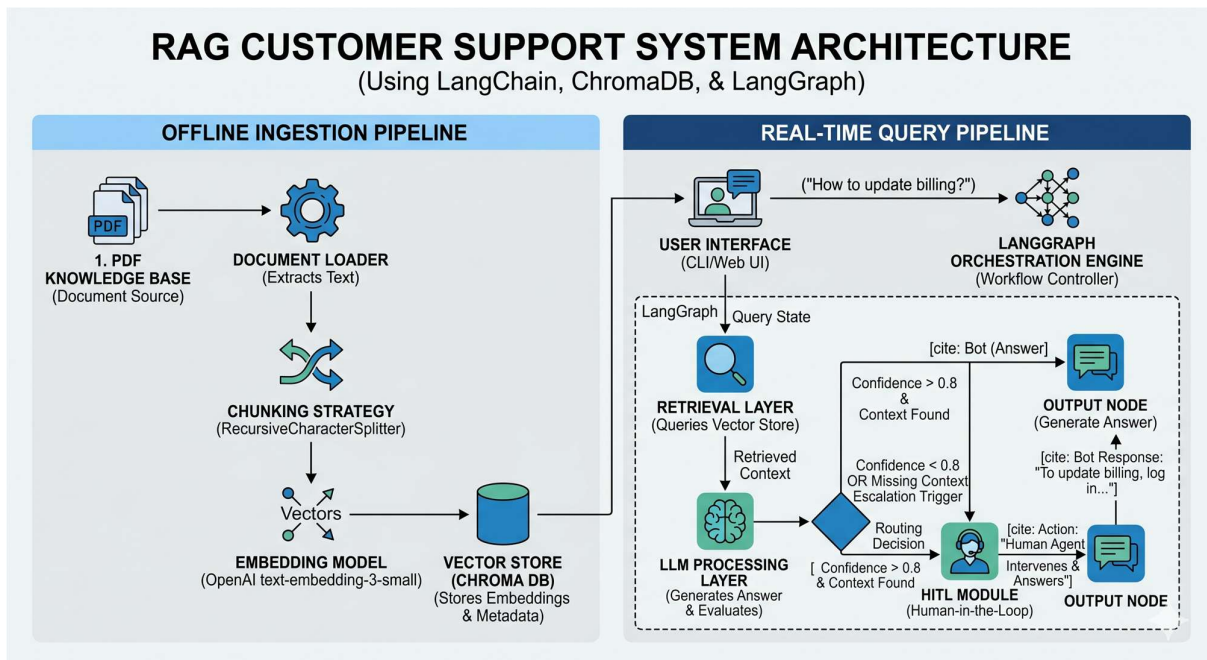
High-Level Design (HLD)

1. System Overview

Problem Definition: Customer support teams often face high volumes of repetitive queries that rely on static knowledge bases. Traditional chatbots lack the ability to retrieve specific document context or gracefully handle unknown queries, leading to poor user experiences or hallucinations.

Scope of the System: This project designs a Retrieval-Augmented Generation (RAG) customer support assistant. The system processes a PDF knowledge base, retrieves context using embeddings, and utilizes a LangGraph-based workflow engine to orchestrate routing. Crucially, the system includes a Human-in-the-Loop (HITL) escalation path for queries that lack confidence or context.

2. Architecture Diagram Description



The system architecture is divided into two primary pipelines:

- **Offline Ingestion Pipeline:** Handles the extraction, chunking, and embedding of the PDF document into the vector database.
- **Real-Time Query Pipeline:** Orchestrated by LangGraph, this pipeline manages user input, retrieval, LLM processing, intent routing, and HITL escalation.

3. Component Description

- **Document Loader:** Extracts raw text data from the provided PDF knowledge base.
- **Chunking Strategy:** Splits the extracted text into manageable, semantically meaningful segments (e.g., using a recursive character text splitter) to optimize context window limits.
- **Embedding Model:** Transforms the text chunks into dense vector representations. (e.g., OpenAI text-embedding-3-small).
- **Vector Store:** A ChromaDB instance used to store and index the vector embeddings for rapid similarity search.
- **Retriever:** The query interface that converts user input into an embedding and fetches the top-K most similar chunks from ChromaDB.
- **LLM:** The core generative engine responsible for synthesizing answers from retrieved context and evaluating its own confidence (e.g., GPT-4o-mini).
- **Graph Workflow Engine:** Powered by LangGraph, this orchestrates the stateful flow of data between processing nodes and controls the application logic.
- **Routing Layer:** A conditional logic node within the graph that evaluates intent and confidence to decide whether to output an answer or escalate.
- **HITL Module:** A fallback node that pauses the workflow, alerts a human agent for manual intervention, and injects the human response back into the user output.

1. Data Flow & Query Lifecycle

The data flow operates in distinct phases:

- **Ingestion:** PDF → Text Extraction → Chunking → Embedding → Stored in ChromaDB.
- **Execution (Query Lifecycle):**
 - **Input:** User submits a query via the UI.
 - **State Creation:** LangGraph initializes a state object containing the query.
 - **Retrieval:** The query is embedded and matched against ChromaDB to pull context.
 - **Evaluation:** The LLM receives the query + context and generates a drafted answer along with a confidence score.
 - **Routing:**
 - If Confidence > 0.85 & Context is present → Route to Output Node.
 - If Confidence < 0.85 OR Context is missing → Route to HITL Module.
 - **Output:** The final answer (bot-generated or human-provided) is returned to the UI.

2. Technology Choices

- **Vector Store - ChromaDB:** Selected for its lightweight, open-source nature, and seamless integration with Python-based LLM frameworks. It runs efficiently in memory for rapid prototyping.
- **Orchestration - LangGraph:** Chosen for its stateful, graph-based execution model. Unlike linear chains, LangGraph allows for cyclical flows and robust conditional routing, which is mandatory for implementing reliable HITL pausing and resuming.
- **LLM Choice:** OpenAI GPT-4o-mini (or equivalent). Selected for its balance of cost-efficiency, low latency, and high instruction-following capabilities required for evaluating confidence scores.

3. Scalability Considerations

- **Handling Large Documents:** As the PDF scales, local chunking may bottleneck memory. The chunking module can be optimized using parallel processing. Furthermore, ChromaDB can be migrated from an in-memory instance to a persistent client/server model.
- **Increasing Query Load:** LangGraph is stateless between node executions, meaning the query pipeline can be deployed on serverless architecture (like AWS Lambda) or containerized via Docker to auto-scale horizontally with incoming traffic.
- **Latency Concerns:** To mitigate latency during retrieval and generation, semantic caching (e.g., Redis) can be introduced to instantly return answers for frequently asked questions without hitting the LLM